Three Week Presentation

Anna Hutter

Thursday 3rd July, 2025

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
   - Lagrange basis functions
   - FEM Vector
5. Next Steps

Outline

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
5. Next Steps

Goal

Full Maxwell solver in IPPL

- FDTD solver exists, only 2nd order
- Finite Elemente Methods Maxwell solver
  - Allows for higher accuracy
  - Matrix free implementation allows for similar memory usage
  - Only increases the computational load on the GPU

Outline

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
5. Next Steps

Finite Element Method (FEM)

Consider the strong form of a PDE (example: Poisson equation):

$$-\Delta u = f \quad \text{in } \Omega, \quad u = 0 \text{ on } \partial\Omega$$

The Variational (weak) form: Find $u \in V = H_1^0(\Omega)$ such that

$$a(u, v) = \int_\Omega \nabla u \cdot \nabla v \, dx = \int_\Omega f v \, dx = l(v) \quad \forall v \in V$$

FEM approximation:

- Discretize the domain in to a mesh $\mathcal{M}$ with elements K: $\overline{\Omega} = \bigcup_{e=1}^{N_E} K_e$
- Choose finite-dimensional subspace $V_h \subset V$
- Find $u_h \in V_h$ such that: $a(u_h, v_h) = l(v_h) \quad \forall v_h \in V_h$

This leads to the system of equations $A\mathbf{x} = \mathbf{b}$ that can be solved matrix free

Lagrange Finite Elements

- V_h can be chosen as the **Lagrange space** of degree p:

$$\mathcal{S}_p^0(\mathcal{M}) = \{v_h \in C^0(\overline{\Omega}) : v_h|_K \in \mathbb{P}_p(K_e),\ \forall K_e \in \mathcal{M}\}$$

with the polynomials of degree $\leq$ p on element K:

$$\mathbb{P}_p(K) := \{x \in K \mapsto \sum_{\alpha \in \mathbb{N}_0^d, |\alpha| \leq p} c_\alpha x^\alpha,\ c_\alpha \in \mathbb{R}\}$$

- The functions in V_h are fully determined by their values at a finite set of degrees of freedom (DOFs), associated with vertices, edges, faces and volumes.
- A basis for V_h is given by $\{b_N^i\}_{i=1}^N$, the Lagrange basis functions satisfying $b_N^i(\mathbf{x}_j) = \delta_{ij}$, $N =$ number of DOFs

From Variational Problem to Linear System

We discretize u:

$$u_h = \sum_{j=1}^{N} \mu_j b_N^j$$

Inserting this into the variational formulation:

$$a(u_h, v_h) = l(v_h) \quad \forall v_h \in V_h$$

$$\Rightarrow \quad \sum_{j=1}^{N} \mu_j a(b_N^j, b_N^i) = l(b_N^i), \quad i = 1, \ldots, N$$

which can be written as the linear system

$$A\mu = \mathbf{b}$$

where$A_{ij} = a(b_N^j, b_N^i), \quad b_i = l(b_N^i), \quad \mu = (\mu_1, \ldots, \mu_N)^T$

Matrix-Free FEM Solver (in IPPL)

In the matrix-free FEM approach, we avoid assembling the stiffness matrix A explicitly.

- Righthand side information provided on ippl::Field
- Variational formulation defining the operator A

Workflow:

1. **FEM Interface:**
   - Projects the RHS onto the FEM space, now called b.
   - Provides a function $evaluateAx()$ to evaluate $A \cdot x$ matrix-free.
2. **CG:** Solves $A\mu = b$ for the coefficient vector μ using $evaluateAx()$.
3. **FEM Solution:** The solution $u_h \in V_h$ is obtained via:

$$u_h(x) = \sum_{j=1}^{N} \mu_j b_h^j(x)$$

4. **Transfer Back:** The solution is transfered back from the FEM space to the ippl::Field

Outline

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
5. Next Steps

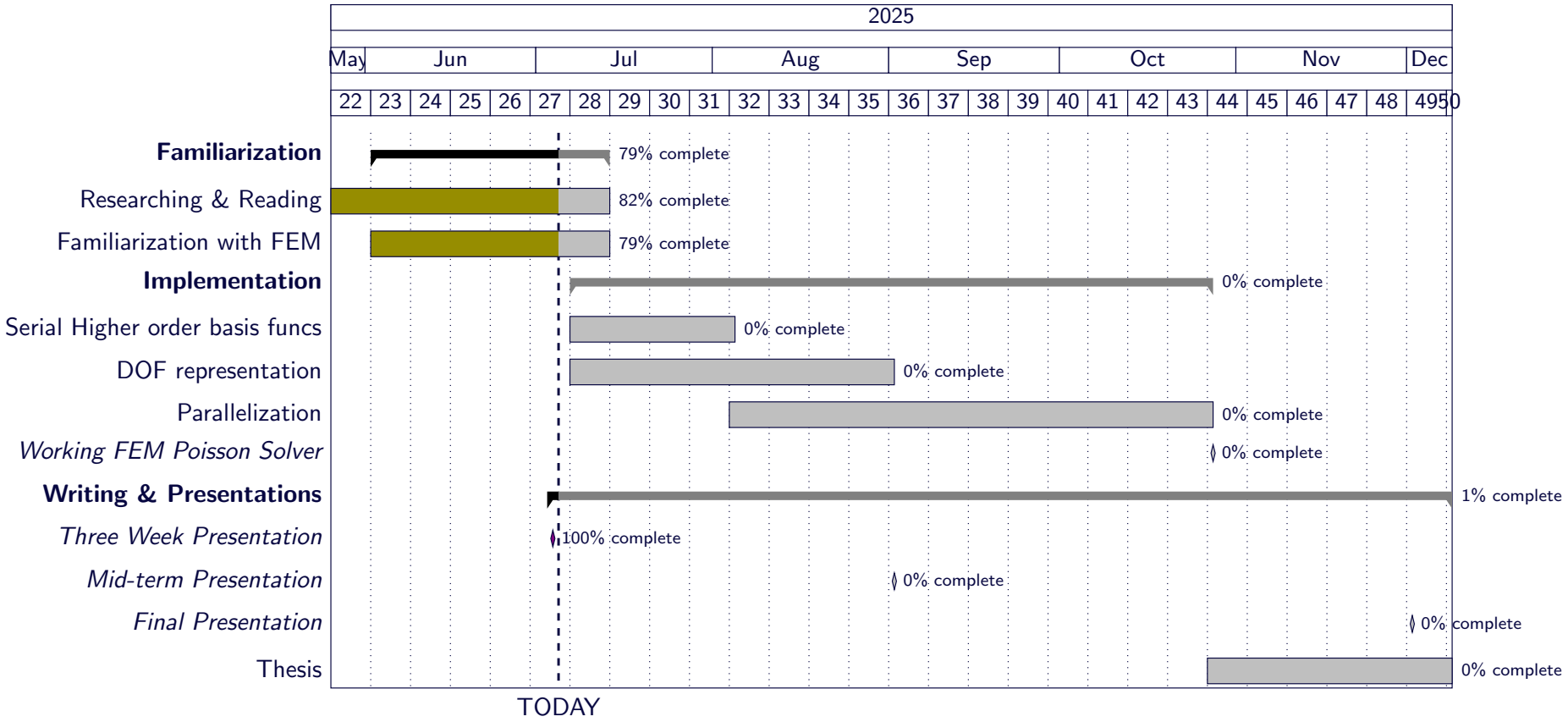
What is already implemented

- A basic **FEM framework** has been introduced:
  - Matrix-free, using structured hexahedral grids
  - Supports first-order Lagrange finite elements
  - `FEMPoissonSolver` provides a working example
- A separate project (nearing completion) has implemented:
  - First-order Nédélec finite elements for Maxwell equations

My Task

- Introduce higher order basis functions on top of the Lagrange space to enable p-refinement.
- Software design of the general higher-order elements in IPPL.
- Correctness studies, performance analysis and scaling studies of the implemented code.

Timeline


2025
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
Familiarization
79% complete
Researching & Reading
82% complete
Familiarization with FEM
79% complete
Implementation
0% complete
Serial Higher order basis funcs
0% complete
DOF representation
0% complete
Parallelization
0% complete
Working FEM Poisson Solver
0% complete
Writing & Presentations
1% complete
Three Week Presentation
100% complete
Mid-term Presentation
0% complete
Final Presentation
0% complete
Thesis
0% complete
TODAY

Outline

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
   - Lagrange basis functions
   - FEM Vector
5. Next Steps

Outline

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
   - Lagrange basis functions
   - FEM Vector
5. Next Steps

Lagrange Basis Functions in 1D

Lagrange basis functions are defined such that $b_N^i(x_j) = \delta_{ij}$ for $i, j = 1, \ldots, N$. The Lagrange basis function $b_N^i(x)$ on interpolation points (DOFs) $x_1, \ldots, x_N$ is given by:

$$b_N^i(x) := \prod_{\substack{j=1 \\ j \neq i}}^{N} \frac{x - x_j}{x_i - x_j}, \quad i = 1, \ldots, N$$

Their gradient is:

$$\frac{d}{dx} b_N^i(x) = \sum_{\substack{k=1 \\ k \neq i}}^{N} \left(\prod_{\substack{j=1 \\ j \neq i, j \neq k}}^{N} \frac{x - x_j}{x_i - x_j} \cdot \frac{1}{x_i - x_k} \right)$$

Tensor Product Basis Functions

In higher dimensions, basis functions can be constructed as tensor products of 1D basis functions.

Let $\{b_N^{i_k}(x_k)\}_{i_k=1}^N$ be a basis in the x_k-direction, for $k = 1, \ldots, d$.

Then the tensor product basis functions for dimension d are:

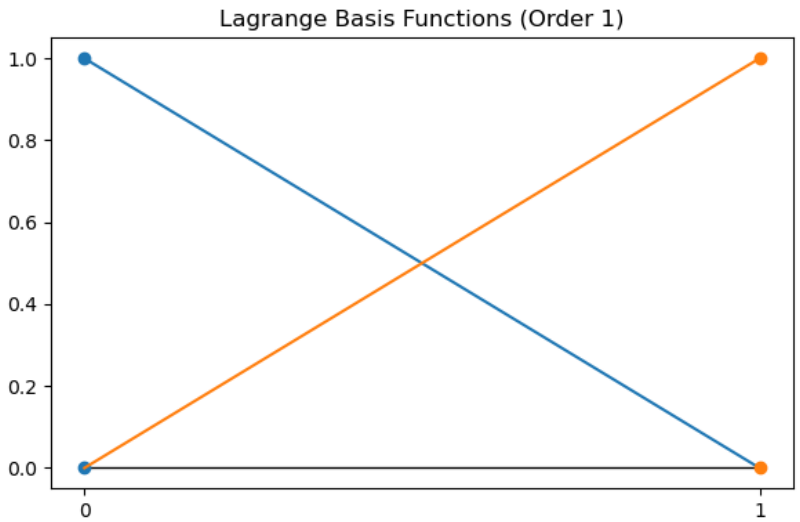
$$b_N^i(x) = \prod_{k=1}^{d} b_N^{i_k}(x_k), \quad \text{where } x = (x_1, \ldots, x_d),\ i = (i_1, \ldots, i_d)$$

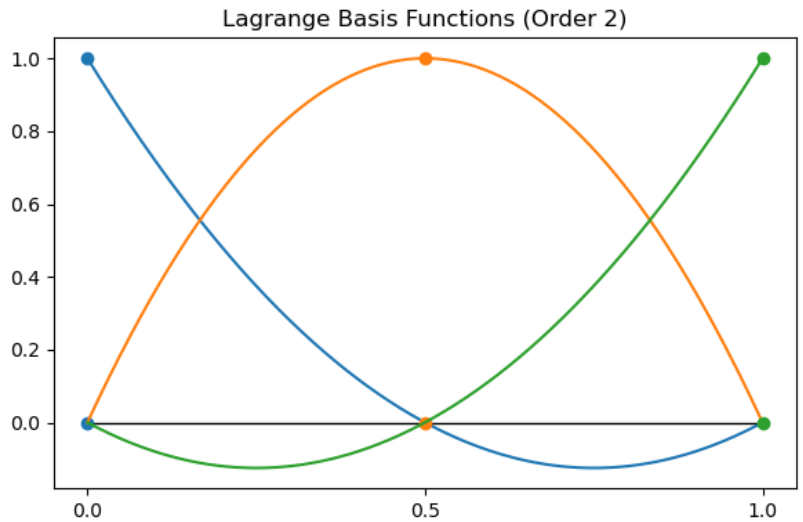
Each basis function is associated with a multi-index i and corresponds to a degree of freedom located at the tensor product of the 1D nodes.

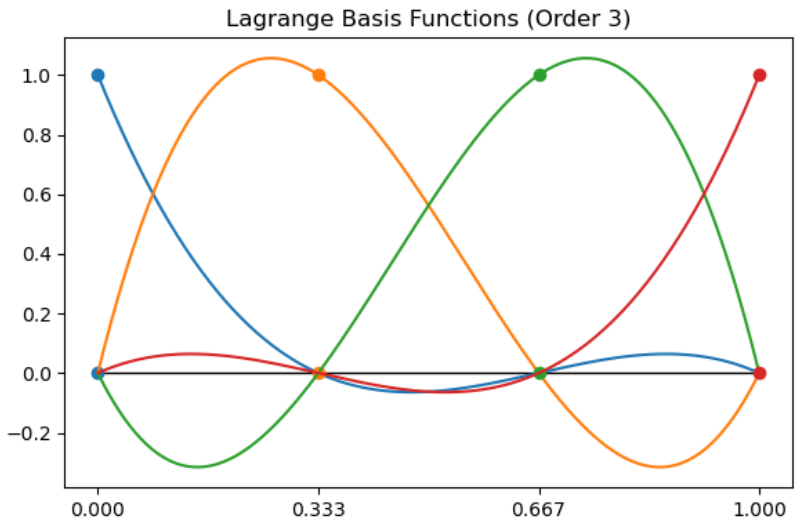
Gradient of Tensor Product Basis Functions

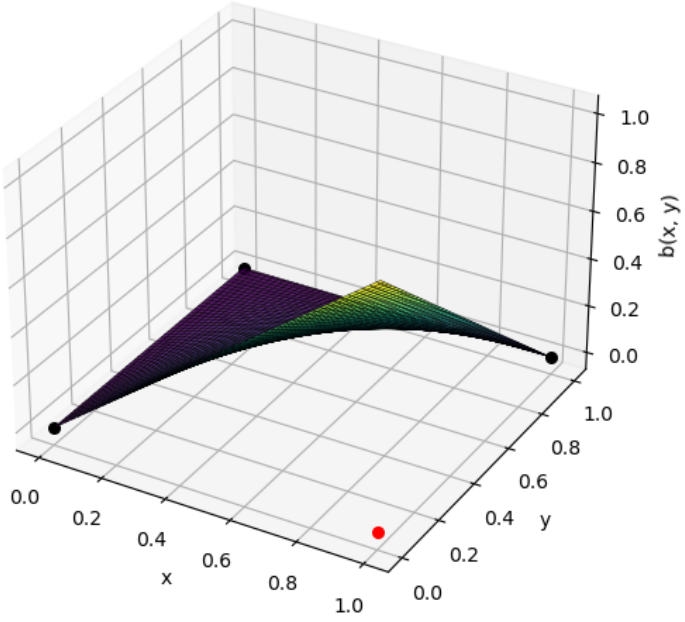
The gradient of a tensor product basis function in d dimensions is given by applying the product rule in each direction:

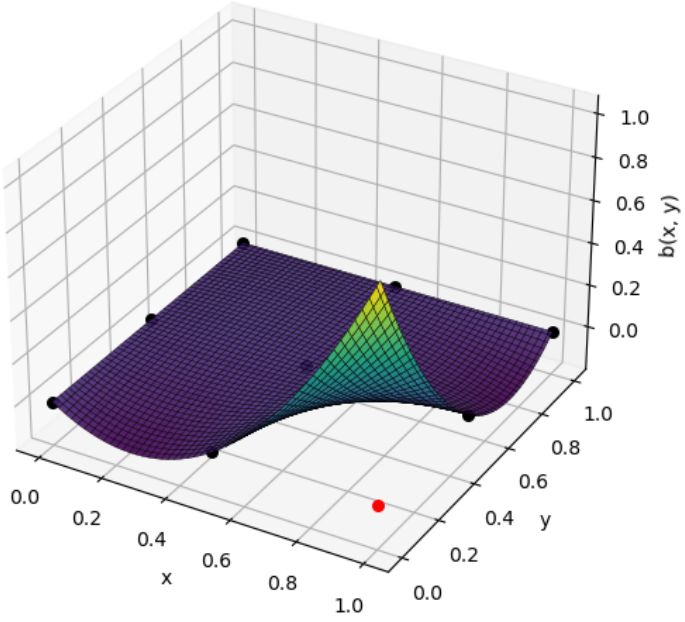
$$\nabla b_N^i(x) = \begin{pmatrix} \left(\frac{d}{dx_1} b_N^{i_1}(x_1)\right) \cdot \prod_{k=2}^{d} b_N^{i_k}(x_k) \\ b_N^{i_1}(x_1) \cdot \left(\frac{d}{dx_2} b_N^{i_2}(x_2)\right) \cdot \prod_{k=3}^{d} b_N^{i_k}(x_k) \\ \vdots \\ \prod_{k=1}^{d-1} b_N^{i_k}(x_k) \cdot \left(\frac{d}{dx_d} b_N^{i_d}(x_d)\right) \end{pmatrix}$$

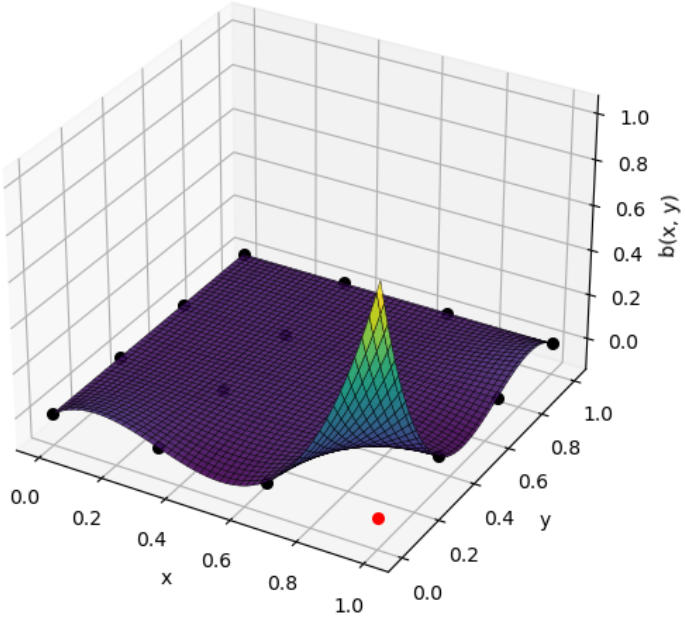

Lagrange Basis Functions (Order 1)
1.0
0.8
0.6
0.4
0.2
0.0
0
1


Lagrange Basis Functions (Order 2)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.5
1.0


Lagrange Basis Functions (Order 3)
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0.000
0.333
0.667
1.000


b(x, y)
x
y
0.0
0.2
0.4
0.6
0.8
1.0

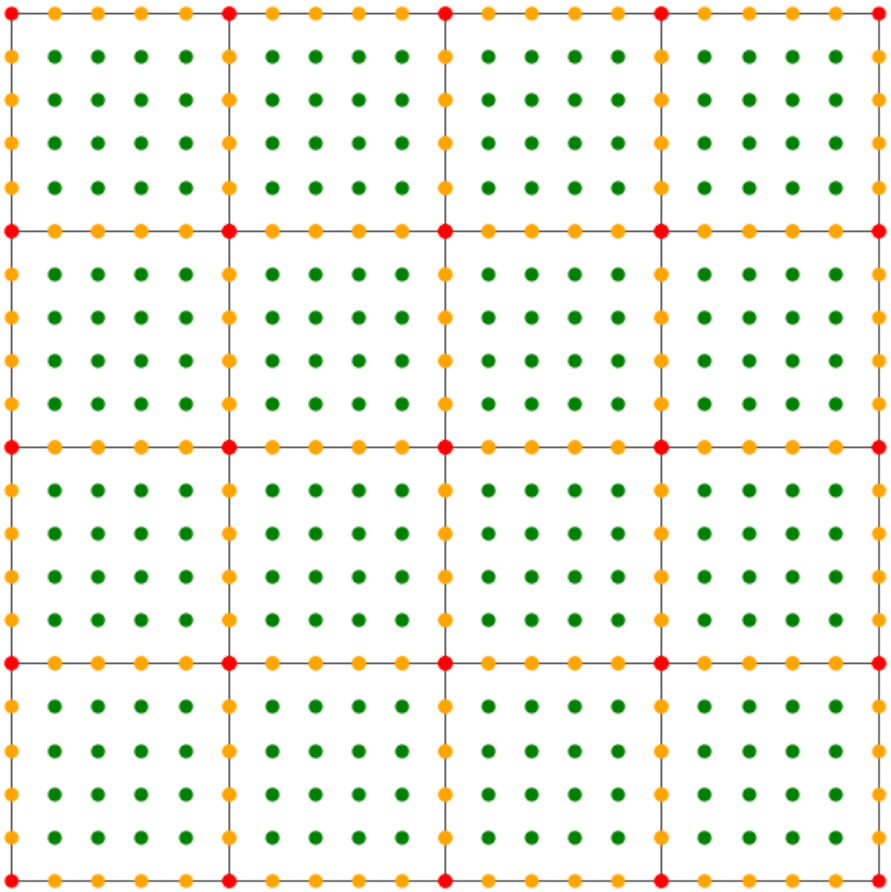

b(x, y)
x
y
0.0
0.2
0.4
0.6
0.8
1.0

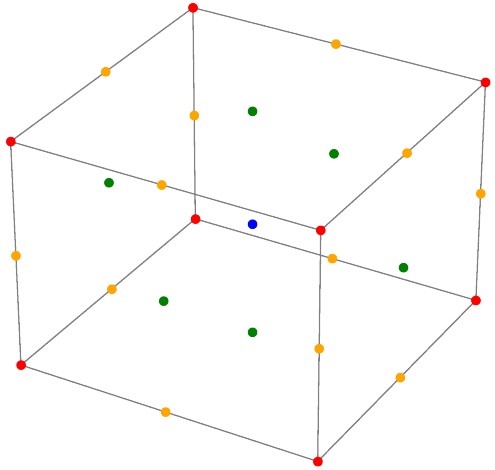

b(x, y)
x
y
0.0
0.2
0.4
0.6
0.8
1.0

Outline

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
   - Lagrange basis functions
   - FEM Vector
5. Next Steps

Representing additional DOFs





The `FEMVector`

- Represents all DOFs using a 1D `Kokkos::View`.
- Enables parallelization via MPI halo exchange:
  - Based on index lists: `neighbors`, `sendIdxs`, `recvIdxs`.
  - Uses `fillHalo()`, `accumulateHalo()`, and `setHalo()`.
- **Advantages**:
  - Works for all FEM spaces.
  - Independent of mesh geometry and layout.
- **Drawback**:
  - Requires custom DOF mapping and halo setup for each FEM space.

Is there an alternative data structure which simplifies the halo exchange?

Outline

1. Goal
2. FEM Introduction
3. Project and Timeline
4. Implementation
5. Next Steps

Next Steps

- Currently working on: Choosing representation for DOFs
- Next steps:
  - Implement 1D serial version higher order Lagrange elements
  - Going to 2D and 3D
  - Implement helo exchange between multiple ranks

Write unit tests for all the completed implementations.

References

[Hiptmair NumCSE] R. Hiptmair. Numerical Methods for Computational Science and Engineering. 2019. URL: https://www.sam.math.ethz.ch/ grsam/NCSE20/NumCSE_Lecture_Document.pdf

[Hiptmair NumPDE] R. Hiptmair. Numerical Methods for (Partial) Differential Equations. 2022. URL: https://www.sam.math.ethz.ch/ grsam/NUMPDEFL/NUMPDE.pdf